ME-429-101
Controls and Instrumentation Lab
Lab 10: Piezoelectric Accelerometer

Josh Fowler, Tyler Kendrick, Kanan Aljeneede

11/18/18

Author Contributions

Josh Fowler (author percent%): author tasks
Tyler Kendrick (author percent%): author tasks
Kanan Aljeneede (author percent%): author tasks

Statement of Quality

This Report is an original work of the above mentioned authors.

Author	Date
Author	Date
Author	Date

Introduction

Theory and Methods

Results

Conclusions and Recomendations

Appendix

A : Collected Data

Table 1: your caption[1]

0.0	0.617821
0.0010000000000000009	0.539514
0.0020000000000000018	0.47404
0.0030000000000000027	0.295054
0.0040000000000000036	0.40034
0.0050000000000000044	0.503651
. . . 3.752	. . . -0.00139203
3.753	-0.00139203
3.754	-0.00139203
3.755	-0.00139203
3.756	-0.00172105
3.757	-0.00139203

Table 2: your caption[2]

[1]Due to the size of this data set, this table has been abbreviated
[2]Due to the size of this data set, this table has been abbreviated

0.0	0.633614
0.0010000000000000009	0.25952
0.0020000000000000018	0.137125
0.0030000000000000027	0.0473028
0.0040000000000000036	0.235501
0.0050000000000000044	0.242082
. . . 3.336	. . . -0.00172105
3.337	-0.00172105
3.338	-0.00139203
3.339	-0.00172105
3.34	-0.00139203
3.341	-0.00139203

Table 3: your caption[3]

0.0	0.180884
0.0010000000000000009	0.0976426
0.0020000000000000018	0.0492769
0.0030000000000000027	0.0463157
0.0040000000000000036	0.0831658
0.0050000000000000044	0.105539
. . . 3.205	. . . -0.00172105
3.2060000000000004	-0.00237908
3.207	-0.00139203
3.208	-0.00237908
3.2089999999999996	-0.00237908
3.21	-0.0027081

Table 4: your caption[4]

[3]Due to the size of this data set, this table has been abbreviated
[4]Due to the size of this data set, this table has been abbreviated

0.0	0.365135
0.0010000000000000009	0.23254
0.0020000000000000018	0.0890882
0.0030000000000000027	0.224973
0.0040000000000000036	0.222012
0.0050000000000000044	0.288802
. . . 3.396	. . . -0.00402418
3.3970000000000002	-0.00369516
3.398	-0.00303712
3.399	-0.00303712
3.4	-0.00369516
3.4010000000000002	-0.00303712

Table 5: your caption[5]

0.0	0.0036591
0.25	0.00248926
0.5	5.37e-05
0.75	6.04e-05
1.0	5.42e-05
1.25	8.98e-05
. . . 38.5	. . . 0.00011908
38.75	0.000160297
39.0	0.000129396
39.25	0.000106692
39.5	0.000217568
39.75	0.00031329

Table 6: your caption[6]

[5]Due to the size of this data set, this table has been abbreviated
[6]Due to the size of this data set, this table has been abbreviated

0.0	0.00380653
0.25	0.00277308
0.5	0.00014633
0.75	9.78e-05
1.0	5.02e-05
1.25	7.68e-05
. . . 38.5	. . . 0.00062031
38.75	0.000626422
39.0	0.000489179
39.25	0.000491637
39.5	0.000667836
39.75	0.000771899

Table 7: your caption[7]

0.0	0.00412772
0.25	0.00301049
0.5	0.000124944
0.75	8.87e-05
1.0	8.95e-05
1.25	0.00010309
. . . 38.25	. . . 0.000408575
38.5	0.000302626
38.75	0.000199188
39.0	0.000162791
39.25	0.000214143
39.5	0.000280749

Table 8: your caption[8]

[7]Due to the size of this data set, this table has been abbreviated
[8]Due to the size of this data set, this table has been abbreviated

0.0	0.00391427
0.25	0.00284933
0.5	0.000129389
0.75	8.74e-05
1.0	5.12e-05
1.25	6.36e-05
. . . 38.25	. . . 0.00033297
38.5	0.000282793
38.75	0.000271477
39.0	0.000230445
39.25	0.00025218
39.5	0.000319506

B Equations

C : Pictures and Graphs

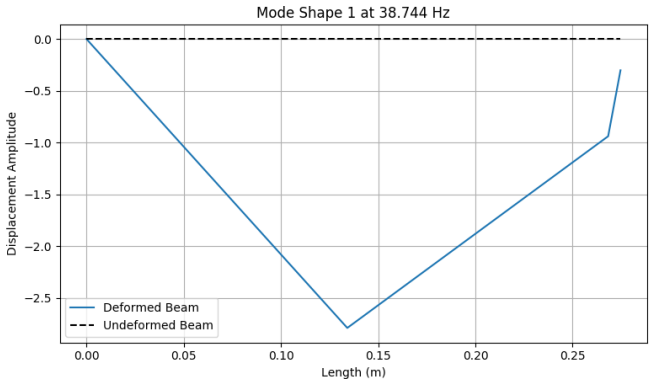

Figure 1: your caption

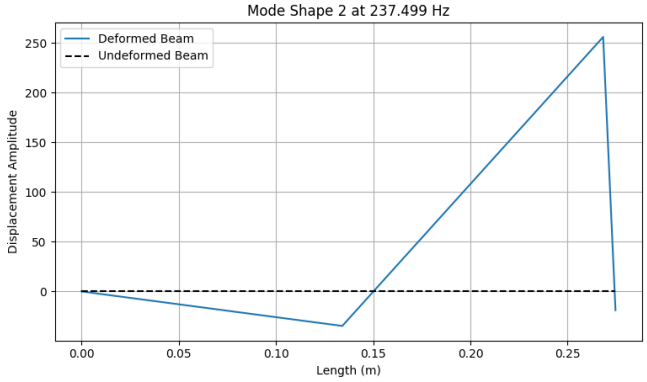

Figure 2: your caption

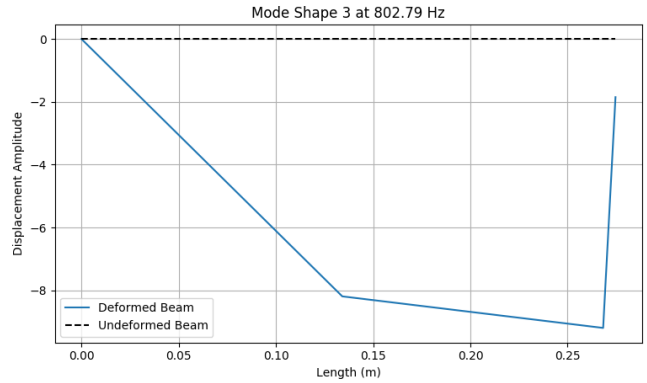

Figure 3: your caption

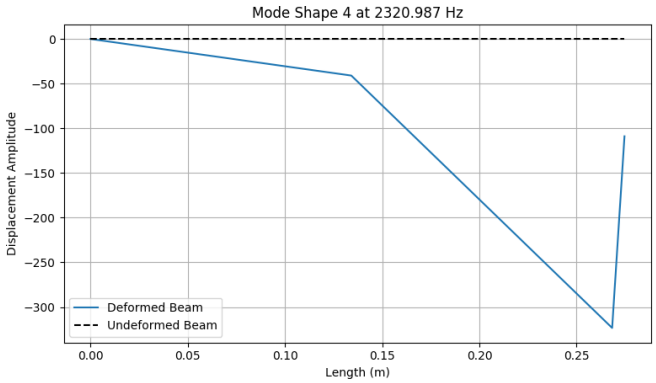

Figure 4: your caption

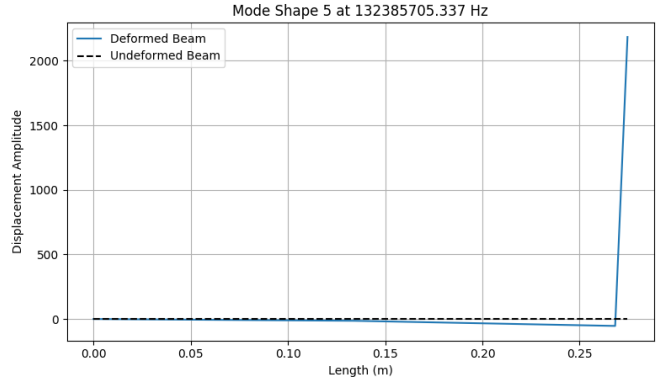

Figure 5: your caption

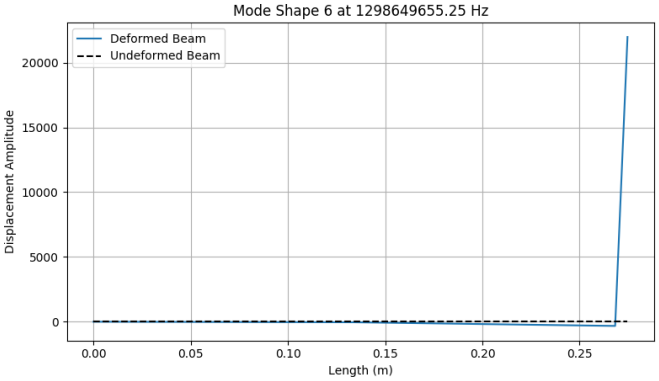

Figure 6: your caption

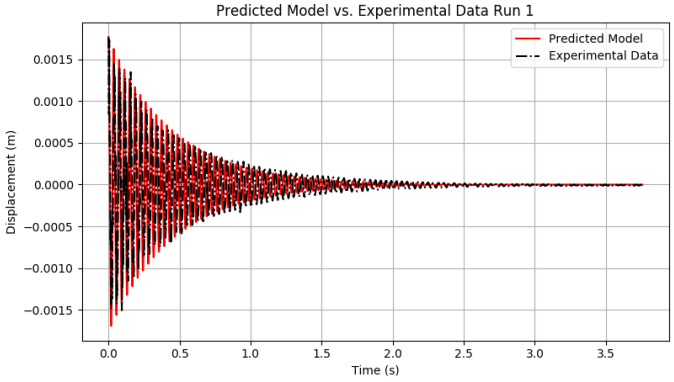

Figure 7: your caption

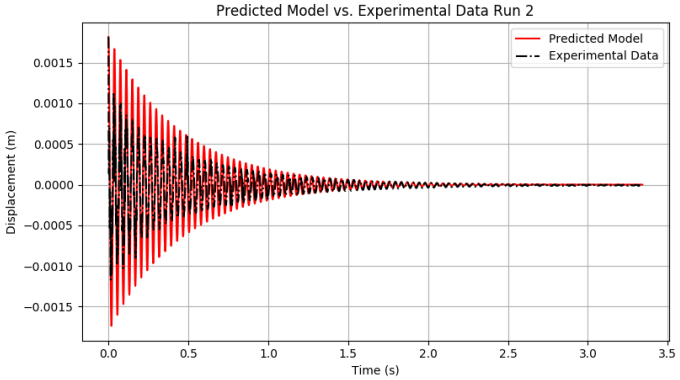

Figure 8: your caption

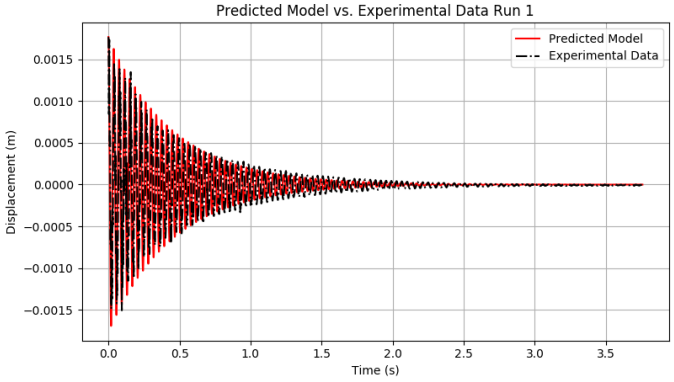

Figure 9: your caption

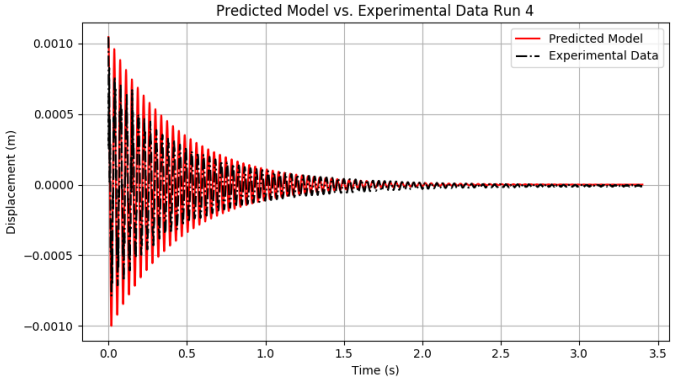

Figure 10: your caption

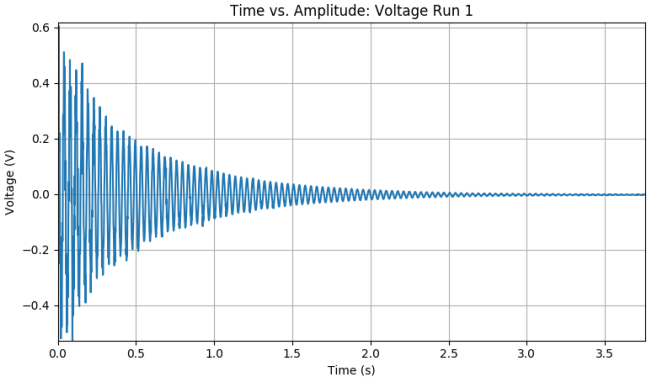

Figure 11: your caption

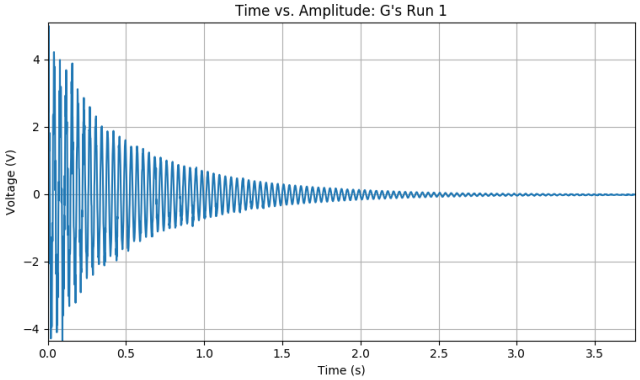

Figure 12: your caption

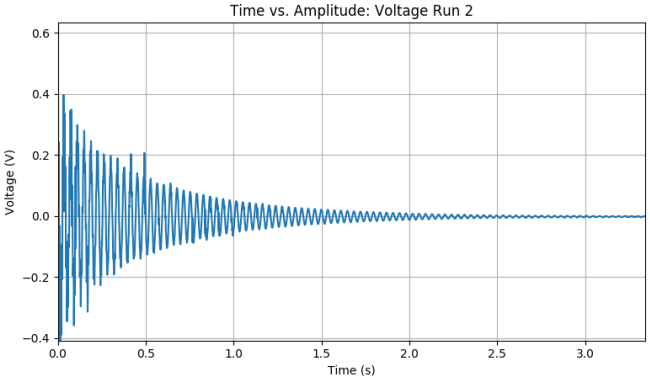

Figure 13: your caption

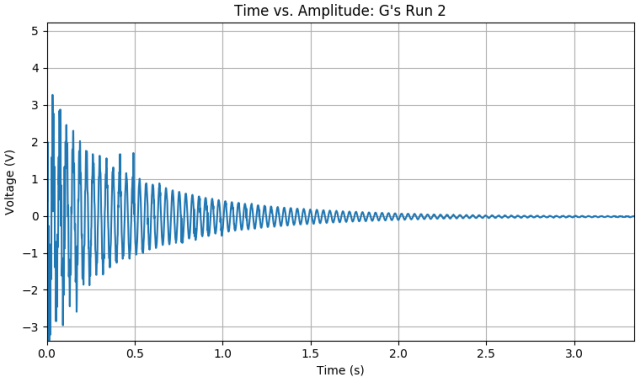

Figure 14: your caption

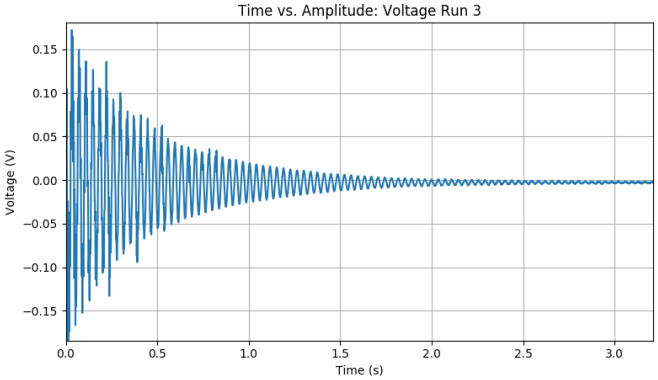

Figure 15: your caption

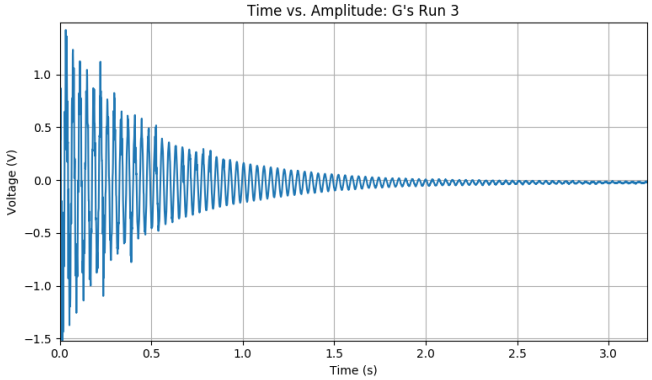

Figure 16: your caption

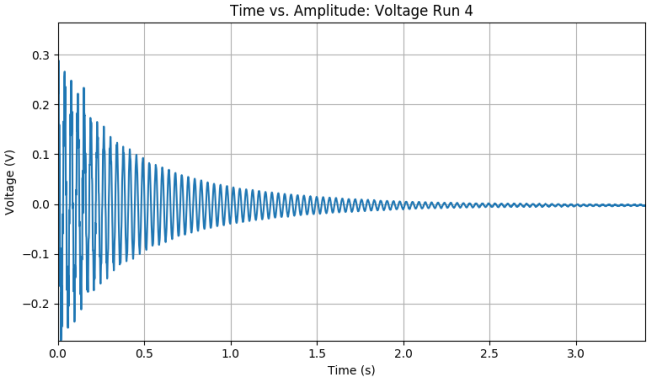

Figure 17: your caption

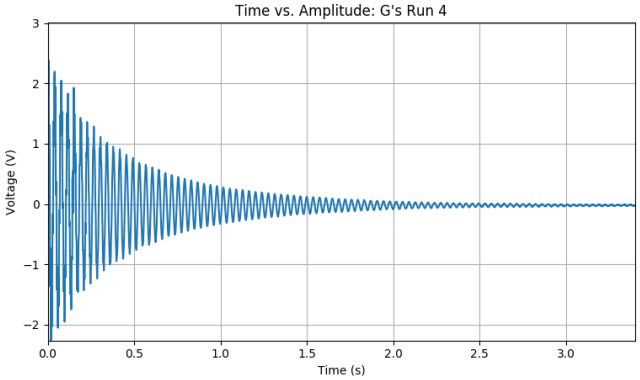

Figure 18: your caption

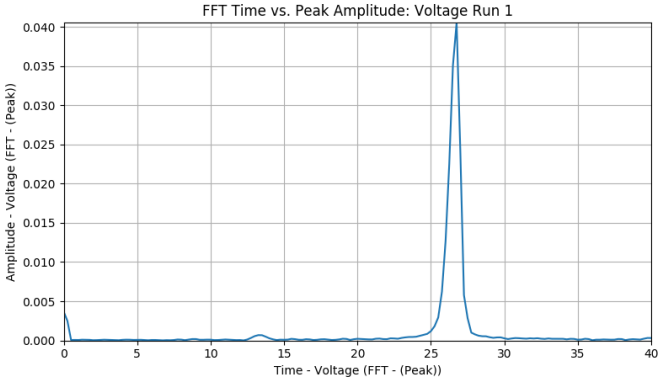

Figure 19: your caption

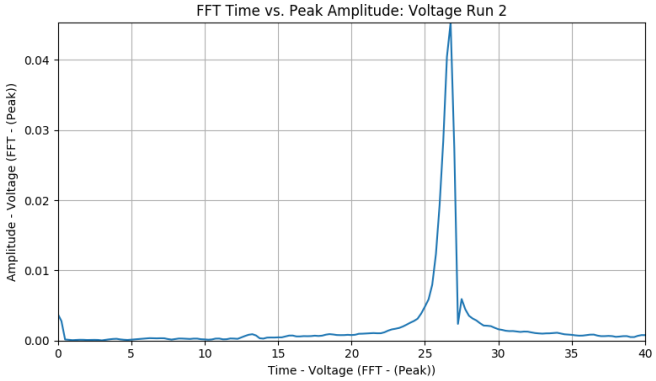

Figure 20: your caption

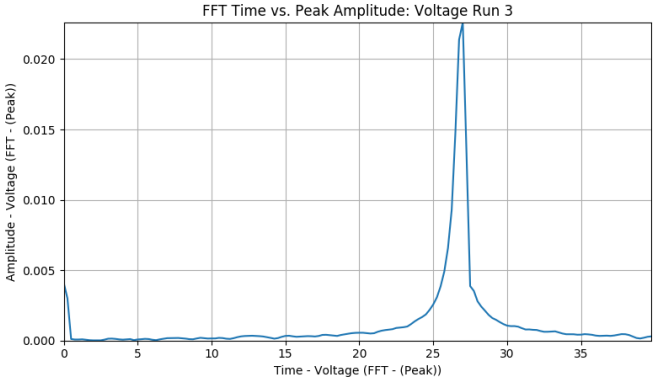

Figure 21: your caption

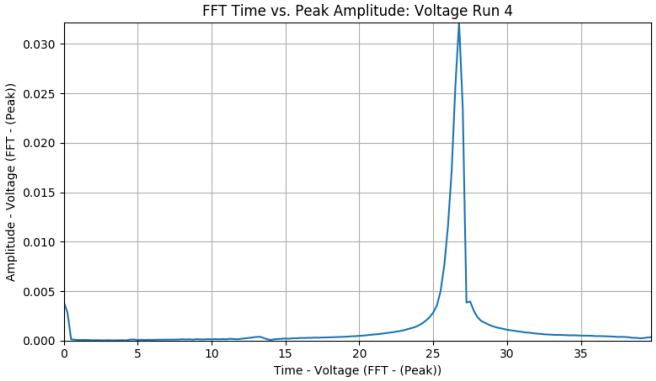

Figure 22: your caption

D : Python Code

```
# -*- coding: utf-8 -*-
"""
Lab 10: Piezoelectric Accelerometer
Determining the Flexural Stiffness of a Cantilever Beam:

    Finite Element Formulation
                                                 _______   | disp
    /|                                          |Acc.|    |
    /|------------------------------------------|Beam|    |
    /| Beam Elem.1       | Beam Elem. 2         |E. 3|    |
    /|-------------------------------------------------|   |
    /|                                                /    v
                                                rot  /
                                                    /
                                               <---/
"""
import numpy as np
import matplotlib.pyplot as plt
import os
import scipy.linalg as lin
import Latex_Tools.starter_file_generator as L
import My_Scripts.Stats_Practice.Regression as regress
import scipy.optimize as so

clss_code = 'ME-429-101'
clss = 'Controls and Instrumentation Lab'
name = 'Piezoelectric Accelerometer'
lab_number = 'Lab 10:'
title_pic = None
thisfname = 'Lab_10_Analysis.py'
cwd = os.getcwd().replace('\\', '/') + '/'
authors = ['Josh Fowler', 'Tyler Kendrick', 'Kanan Aljeneede']

report = L.lstarter_file_gen(clss_code, clss, name, lab_number
   , title_pic,
                             cwd, thisfname, authors)
report.make_body()

x02 = -1.125*0.0254  # m
x03 = -(7/8)*0.0254  # m
x04 = -1*0.0254  # m
g = 9.81  # gravity m/s^2
gpv = 8.25  # g/mV
Length = 10.8125*0.0254  # m
ltot = 12*0.0254  # m
b = 1.0*0.0254  # width m
```

```
h = (1/8)*0.0254  # thickness m
h2 = (1/4)*0.0254
ltot = 12*0.0254  # total length m
vol = ltot*b*h  # volume m^3
A = b*h  # Cross Sectional Area m^2
A2 = b*h2
rho = (2.7*100**3)/1000  # kg per cm 6061 Al
mass_al = 66.12  # mass grams
mass_al_kg = mass_al/1000  # mass kg
mass_acc = 10.6  # mass grams
mass_acc_kg = mass_acc/1000  # mass kg
density_calc = ((mass_al_kg)/vol)  # calculated density kg/m^3
rho_l = density_calc*A  # linear density kg/m
dens_error = abs(rho - density_calc)  # error between
   densities
effective_mass = (33/140)*(rho_l*Length) + mass_acc_kg  # kg
endl = (1/4)*0.0254
length = Length - endl
rho_l2 = (rho_l*endl + mass_acc_kg)/endl
rho1 = rho_l*length
rho2 = rho_l2*endl
Iz1 = (1/12)*(rho_l*length)*(b*(h**3))  # Moment of Inertia m
   ^4
Iz2 = (1/12)*(mass_acc_kg + rho_l*Length)*((b)*((1/2)*0.0254 +
    h)**3)
Iz = (1.5/10)*Iz2 - (8.5/10)*Iz1


def local_maxima(xval, yval):
    xval = np.asarray(xval)
    yval = np.asarray(yval)

    sort_idx = np.argsort(xval)
    yval = yval[sort_idx]
    gradient = np.diff(yval)
    maxima = np.diff((gradient > 0).view(np.int8))
    return np.concatenate((([0],) if gradient[0] < 0 else ())
       +
                          (np.where(maxima == -1)[0] + 1,) +
                          (([len(yval)-1],) if gradient[-1] >
                             0 else ()))

data = []
data_file = open('Lab_10_data.csv', 'r')
for line in data_file.readlines():
    data_line = line.strip('\n').split(',')
    data.append(data_line)
data = np.asarray(data)
```

```
data_file.close()

run1 = data[:, :4]
run2 = data[:, 4:8]
run3 = data[:, 8:12]
run4 = data[:, 12:16]

run1_tv = run1[1:, 0:2]
run1_fft = run1[1:162, 2:4]
run1_tv = run1_tv.astype(float)
peak1 = max(run1_tv[:, 1])
for i in range(len(run1_tv)):
    if run1_tv[i, 1] == peak1:
        run1_tv = run1_tv[i:, :]
        break
run1_tv = np.asarray([[run1_tv[i, 0] - run1_tv[0, 0], run1_tv[
   i, 1]] for i in range(len(run1_tv))])
run1_fft = run1_fft.astype(float)
run1_pfi = max(run1_fft[:, 1])
for i in range(len(run1_fft[:, 1])):
    if run1_fft[i, 1] == run1_pfi:
        run1_pf = run1_fft[i, 0]*2*np.pi
run1_keq = effective_mass*(run1_pf)**2
run2_tv = run2[1:, 0:2]
run2_fft = run2[1:162, 2:4]
run2_tv = run2_tv.astype(float)
peak2 = max(run2_tv[:, 1])
for i in range(len(run2_tv)):
    if run2_tv[i, 1] == peak2:
        run2_tv = run2_tv[i:, :]
        break
run2_tv = np.asarray([[run2_tv[i, 0] - run2_tv[0, 0], run2_tv[
   i, 1]] for i in range(len(run2_tv))])
run2_fft = run2_fft.astype(float)
run2_pfi = max(run2_fft[:, 1])
for i in range(len(run2_fft)):
    if run2_fft[i, 1] == run2_pfi:
        run2_pf = run2_fft[i, 0]*2*np.pi
run2_keq = effective_mass*(run2_pf)**2
run3_tv = run3[1:, 0:2]
run3_fft = run3[1:161, 2:4]
run3_tv = run3_tv.astype(float)
peak3 = max(run3_tv[:, 1])
for i in range(len(run3_tv)):
    if run3_tv[i, 1] == peak3:
        run3_tv = run3_tv[i:, :]
        break
run3_tv = np.asarray([[run3_tv[i, 0] - run3_tv[0, 0], run3_tv[
```

```
   i, 1]] for i in range(len(run3_tv))])
run3_fft = run3_fft.astype(float)
run3_pfi = max(run3_fft[:, 1])
for i in range(len(run3_fft)):
    if run3_fft[i, 1] == run3_pfi:
        run3_pf = run3_fft[i, 0]*2*np.pi
run3_keq = effective_mass*(run3_pf)**2
run4_tv = run4[1:, 0:2]
run4_fft = run4[1:161, 2:4]
run4_tv = run4_tv.astype(float)
peak4 = max(run4_tv[:, 1])
for i in range(len(run4_tv)):
    if run4_tv[i, 1] == peak4:
        run4_tv = run4_tv[i:, :]
        break
run4_tv = np.asarray([[run4_tv[i, 0] - run4_tv[0, 0], run4_tv[
   i, 1]] for i in range(len(run4_tv))])
run4_fft = run4_fft.astype(float)
run4_pfi = max(run4_fft[:, 1])
for i in range(len(run4_fft)):
    if run4_fft[i, 1] == run4_pfi:
        run4_pf = run4_fft[i, 0]*2*np.pi
run4_keq = effective_mass*(run4_pf)**2
nfs = [run1_pf, run2_pf, run3_pf, run4_pf]
nfmean = np.mean(nfs)
EIs = []

run1_tg = np.asarray([[run1_tv[i, 0], run1_tv[i, 1]*gpv] for i
    in range(len(run1_tv))])
run2_tg = np.asarray([[run2_tv[i, 0], run2_tv[i, 1]*gpv] for i
    in range(len(run2_tv))])
run3_tg = np.asarray([[run3_tv[i, 0], run3_tv[i, 1]*gpv] for i
    in range(len(run3_tv))])
run4_tg = np.asarray([[run4_tv[i, 0], run4_tv[i, 1]*gpv] for i
    in range(len(run4_tv))])

run1_td = np.asarray([[run1_tg[i, 0], (g*effective_mass/
   run1_keq)*run1_tg[i, 1]] for i in range(len(run1_tg))])
run2_td = np.asarray([[run2_tg[i, 0], (g*effective_mass/
   run2_keq)*run2_tg[i, 1]] for i in range(len(run2_tg))])
run3_td = np.asarray([[run3_tg[i, 0], (g*effective_mass/
   run3_keq)*run3_tg[i, 1]] for i in range(len(run3_tg))])
run4_td = np.asarray([[run4_tg[i, 0], (g*effective_mass/
   run4_keq)*run4_tg[i, 1]] for i in range(len(run4_tg))])

run1idx = local_maxima(run1_td[:, 0], run1_td[:, 1])
run2idx = local_maxima(run2_td[:, 0], run2_td[:, 1])
run3idx = local_maxima(run3_td[:, 0], run3_td[:, 1])
```

```
run4idx = local_maxima(run4_td[:, 0], run4_td[:, 1])

decay1 = np.asarray([[run1_td[i, 0], run1_td[i, 1]] for i in
   run1idx])
for i in range(len(decay1)-1, 0, -1):
    if decay1[i, 1] < 0:
        decay1 = np.delete(decay1, i, 0)
decay12 = np.asarray([[decay1[i, 0], np.log(decay1[i, 1])] for
    i in range(len(decay1))])
decay2 = np.asarray([[run2_td[i, 0], run2_td[i, 1]] for i in
   run2idx])
for i in range(len(decay2)-1, 0, -1):
    if decay2[i, 1] < 0:
        decay2 = np.delete(decay2, i, 0)
decay22 = np.asarray([[decay2[i, 0], np.log(decay2[i, 1])] for
    i in range(len(decay2))])
decay3 = np.asarray([[run3_td[i, 0], run3_td[i, 1]] for i in
   run3idx])
for i in range(len(decay3)-1, 0, -1):
    if decay3[i, 1] < 0:
        decay3 = np.delete(decay3, i, 0)
decay32 = np.asarray([[decay3[i, 0], np.log(decay3[i, 1])] for
    i in range(len(decay3))])
decay4 = np.asarray([[run4_td[i, 0], run4_td[i, 1]] for i in
   run4idx])
for i in range(len(decay4)-1, 0, -1):
    if decay4[i, 1] < 0:
        decay4 = np.delete(decay4, i, 0)
decay42 = np.asarray([[decay4[i, 0], np.log(decay4[i, 1])] for
    i in range(len(decay4))])

run1idx = local_maxima(decay1[:, 0], decay1[:, 1])
run2idx = local_maxima(decay2[:, 0], decay2[:, 1])
run3idx = local_maxima(decay3[:, 0], decay3[:, 1])
run4idx = local_maxima(decay4[:, 0], decay4[:, 1])

decay1 = np.asarray([[decay1[i, 0], decay1[i, 1]] for i in
   run1idx])
for i in range(len(decay1)-1, 0, -1):
    if decay1[i, 1] < 0:
        decay1 = np.delete(decay1, i, 0)
decay12 = np.asarray([[decay1[i, 0], np.log(decay1[i, 1])] for
    i in range(len(decay1))])
decay2 = np.asarray([[decay2[i, 0], decay2[i, 1]] for i in
   run2idx])
for i in range(len(decay2)-1, 0, -1):
    if decay2[i, 1] < 0:
        decay2 = np.delete(decay2, i, 0)
```

```
decay22 = np.asarray([[decay2[i, 0], np.log(decay2[i, 1])] for
    i in range(len(decay2))])
decay3 = np.asarray([[decay3[i, 0], decay3[i, 1]] for i in
   run3idx])
for i in range(len(decay3)-1, 0, -1):
    if decay3[i, 1] < 0:
        decay3 = np.delete(decay3, i, 0)
decay32 = np.asarray([[decay3[i, 0], np.log(decay3[i, 1])] for
    i in range(len(decay3))])
decay4 = np.asarray([[decay4[i, 0], decay4[i, 1]] for i in
   run4idx])
for i in range(len(decay4)-1, 0, -1):
    if decay4[i, 1] < 0:
        decay4 = np.delete(decay4, i, 0)
decay42 = np.asarray([[decay4[i, 0], np.log(decay4[i, 1])] for
    i in range(len(decay4))])

damp1, A1 = regress.theil_sen(decay12[:, 0], decay12[:, 1])
damp2, A2 = regress.theil_sen(decay22[:, 0], decay22[:, 1])
damp3, A3 = regress.theil_sen(decay32[:, 0], decay32[:, 1])
damp4, A4 = regress.theil_sen(decay42[:, 0], decay42[:, 1])


damps = [damp1, damp2, damp3, damp4]
As = [A1, A2, A3, A4]
As = [np.exp(A) for A in As]
damp = abs(np.mean(damps))/(2*np.pi)
damp_const = np.sqrt(1 - damp**2)

b = 0
for Natural_Freq in nfs:
    b += 1
    EI = (((Natural_Freq/damp_const)**2))*(Length**3)*
       effective_mass*(1/3)
    EIs.append(EI)
    print('\tFlexural Stiffness Run {}: '.format(b), EI)
EI = np.mean(EIs)
Eal = 69e9
E = EI/Iz
print('\tAverage Flexural Stiffness: ', EI)
print('\n\n')

report.appendix()
report.include_data_abrev(run1_tv, add_section=True)
report.include_data_abrev(run2_tv)
report.include_data_abrev(run3_tv)
report.include_data_abrev(run4_tv)
report.include_data_abrev(run1_fft)
```

```
report.include_data_abrev(run2_fft)
report.include_data_abrev(run3_fft)
report.include_data_abrev(run4_fft)
report.add_section('Equations')

natural_freq = Natural_Freq/damp_const

print('Damped Natural Frequency: {} rad/s, {} Hz'.format(
   Natural_Freq,
      Natural_Freq/(2*np.pi)), '\nNatural Frequency: {} rad/s,
         {} Hz'.format(
              natural_freq, (natural_freq)/(2*np.pi)),
              '\nEI: ', EI, '\nYoung\'s Modulus: {} GPa'.
                 format((EI/Iz)*1e-9))

num_elems = 3
L = Length/(num_elems - 1)
EI1 = EI/(Iz1)
m = (rho_l*L)/420
m2 = (rho_l2*endl)/420
mmat = np.asarray([[156, 22*L, 54, -13*L],
                   [22*L, 4*L**2, 13*L, -3*L**2],
                   [54, 13*L, 156, -22*L],
                   [-13*L, -3*L**2, -22*L, 4*L**2]])

mmat = m*mmat
mmatend = m2*mmat
I1 = (1/3)*(rho1*b*h*(L/2))*(L/2)**2
I2 = (1/3)*(rho2*b*h2*(endl/2))*(endl/2)**2
k = (EI)/L**3
k2 = (EI)/(endl**3)
#k = Eal*Iz1
#k2 = Eal*Iz2
kmat = np.asarray([[12, 6*L, -12*L, 6*L],
                   [6*L, 4*L**2, -6*L, 2*L**2],
                   [-12, -6*L, 12, -6*L],
                   [6*L, 2*L**2, -6*L, 4*L**2]])

kmat = k*kmat
kmatend = k2*kmat

M = np.zeros([2*num_elems + 2, 2*num_elems + 2])
K = np.zeros([2*num_elems + 2, 2*num_elems + 2])

for i in range(num_elems - 1):
    Mge = np.zeros([2*num_elems + 2, 2*num_elems + 2])
    Kge = np.zeros([2*num_elems + 2, 2*num_elems + 2])
    Mge[2*i:2*i + 4, 2*i:2*i + 4] = mmat
```

```
    Kge[2*i:2*i + 4, 2*i:2*i + 4] = kmat
    M += Mge
    K += Kge
Mge = np.zeros([2*num_elems + 2, 2*num_elems + 2])
Kge = np.zeros([2*num_elems + 2, 2*num_elems + 2])
Mge[2*(num_elems - 1):2*(num_elems - 1) + 4, 2*(num_elems - 1)
   :2*(num_elems - 1) + 4] = mmatend
Kge[2*(num_elems - 1):2*(num_elems - 1) + 4, 2*(num_elems - 1)
   :2*(num_elems - 1) + 4] = kmatend
M += Mge
K += Kge


restrained_dofs = [1, 0]

# remove the fixed degrees of freedom
for dof in restrained_dofs:
        for i in [0,1]:
                M = np.delete(M, dof, axis=i)
                K = np.delete(K, dof, axis=i)


eigvals, eigvecs = lin.eigh(K, M)

print('\n\n')
print('FEA Results:\n\n')
print('Natural Frequencies:')
for i in range(len(eigvals)):
    w = np.sqrt(eigvals[i])
    print('\t{}: {} rad/s, {} Hz'.format(i + 1, w, w/(2*np.pi)
       ))
print('\n\n')
print('Modal Shapes: \n')
figures = []
i = 0
for natural_freqs in eigvals:
    eigs = eigvecs[i]
    for z in range(int(len(eigvecs[i])/2)):
        eigs = np.delete(eigs, -z - 1, 0)
    length_vec = [0, length/2, length, length + endl]
    shape = [0]
    for e in eigs:
        shape.append(e)
    plt.figure()
    plt.title('Mode Shape {} at {} Hz'.format(i + 1, round(np.
       sqrt(natural_freqs)/(2*np.pi), 3)))
    plt.plot(length_vec, shape, label='Deformed Beam')
    plt.plot(length_vec, [0 for k in length_vec], '--k', label
```

```
        ='Undeformed Beam')
    plt.xlabel('Length (m)')
    plt.ylabel('Displacement Amplitude')
    plt.grid(True)
    plt.legend()
    fig_name_mode_shape = 'Mode_shape_{}.png'.format(i+1)
    figures.append(fig_name_mode_shape)
    plt.savefig(fig_name_mode_shape)
    plt.show()
    i += 1


def sinusoidal_decay(int_disp, damp, damp_natural_freq, tvec):
    response = []
    for t in tvec:
        A = int_disp
        B = np.exp(-damp*t)
        C = np.cos(damp_natural_freq*t)
        response.append(A*B*C)
    return response


y1 = sinusoidal_decay(run1_td[0, 1], -damp1, run1_pf, run1_td
    [:, 0])
y2 = sinusoidal_decay(run2_td[0, 1], -damp2, run2_pf, run2_td
    [:, 0])
y3 = sinusoidal_decay(run3_td[0, 1], -damp3, run3_pf, run3_td
    [:, 0])
y4 = sinusoidal_decay(run4_td[0, 1], -damp4, run4_pf, run4_td
    [:, 0])

plt.figure()
plt.plot(run1_td[:, 0], y1, 'r', label='Predicted Model')
plt.plot(run1_td[:, 0], run1_td[:, 1], 'k-.', label='
    Experimental Data')
plt.grid(True)
plt.title('Predicted Model vs. Experimental Data Run 1')
plt.xlabel('Time (s)')
plt.ylabel('Displacement (m)')
plt.legend()
figname = 'PEtd1.png'
plt.savefig(figname)
figures.append(figname)
plt.show()

plt.figure()
plt.plot(run2_td[:, 0], y2, 'r', label='Predicted Model')
plt.plot(run2_td[:, 0], run2_td[:, 1], 'k-.', label='
```

```
   Experimental Data')
plt.grid(True)
plt.title('Predicted Model vs. Experimental Data Run 2')
plt.xlabel('Time (s)')
plt.ylabel('Displacement (m)')
plt.legend()
figname2 = 'PEtd2.png'
plt.savefig(figname2)
figures.append(figname2)
plt.show()

plt.figure()
plt.plot(run3_td[:, 0], y3, 'r', label='Predicted Model')
plt.plot(run3_td[:, 0], run3_td[:, 1], 'k-.', label='
   Experimental Data')
plt.grid(True)
plt.title('Predicted Model vs. Experimental Data Run 3')
plt.xlabel('Time (s)')
plt.ylabel('Displacement (m)')
plt.legend()
figname3 = 'PEtd3.png'
plt.savefig(figname3)
figures.append(figname)
plt.show()

plt.figure()
plt.plot(run4_td[:, 0], y4, 'r', label='Predicted Model')
plt.plot(run4_td[:, 0], run4_td[:, 1], 'k-.', label='
   Experimental Data')
plt.grid(True)
plt.title('Predicted Model vs. Experimental Data Run 4')
plt.xlabel('Time (s)')
plt.ylabel('Displacement (m)')
plt.legend()
figname4 = 'PEtd4.png'
plt.savefig(figname4)
figures.append(figname4)
plt.show()

xlabel_tv = 'Time (s)'
ylabel_tv = 'Voltage (V)'
ylabel_tg = 'G\'s'
xlabel_fft = run1[0, 2]
ylabel_fft = run1[0, 3]
tv_title = 'Time vs. Amplitude: Voltage'
fft_title = 'FFT Time vs. Peak Amplitude: Voltage'
tg_title = 'Time vs. Amplitude: G\'s'
```

```
plt.figure()
plt.plot(run1_tv[:, 0], run1_tv[:, 1])
plt.title(tv_title + ' Run 1')
plt.xlabel(xlabel_tv)
plt.ylabel(ylabel_tv)
plt.xlim([min(run1_tv[:, 0]), max(run1_tv[:, 0])])
plt.ylim([min(run1_tv[:, 1]), max(run1_tv[:, 1])])
plt.grid(True)
tv1 = cwd + 'tv1.png'
figures.append(tv1)
plt.savefig(tv1)
plt.show()

plt.figure()
plt.plot(run1_tv[:, 0], run1_tv[:, 1]*gpv)
plt.title(tg_title + ' Run 1')
plt.xlabel(xlabel_tv)
plt.ylabel(ylabel_tv)
plt.xlim([min(run1_tv[:, 0]), max(run1_tv[:, 0])])
plt.ylim([min(run1_tv[:, 1]*gpv), max(run1_tv[:, 1]*gpv)])
plt.grid(True)
tg1 = cwd + 'tg1.png'
figures.append(tg1)
plt.savefig(tg1)
plt.show()

plt.figure()
plt.plot(run2_tv[:, 0], run2_tv[:, 1])
plt.title(tv_title + ' Run 2')
plt.xlabel(xlabel_tv)
plt.ylabel(ylabel_tv)
plt.xlim([min(run2_tv[:, 0]), max(run2_tv[:, 0])])
plt.ylim([min(run2_tv[:, 1]), max(run2_tv[:, 1])])
plt.grid(True)
tv2 = cwd + 'tv2.png'
figures.append(tv2)
plt.savefig(tv2)
plt.show()

plt.figure()
plt.plot(run2_tv[:, 0], run2_tv[:, 1]*gpv)
plt.title(tg_title + ' Run 2')
plt.xlabel(xlabel_tv)
plt.ylabel(ylabel_tv)
plt.xlim([min(run2_tv[:, 0]), max(run2_tv[:, 0])])
plt.ylim([min(run2_tv[:, 1]*gpv), max(run2_tv[:, 1]*gpv)])
plt.grid(True)
tg2 = cwd + 'tg2.png'
```

```
figures.append(tg2)
plt.savefig(tg2)
plt.show()

plt.figure()
plt.plot(run3_tv[:, 0], run3_tv[:, 1])
plt.title(tv_title + ' Run 3')
plt.xlabel(xlabel_tv)
plt.ylabel(ylabel_tv)
plt.xlim([min(run3_tv[:, 0]), max(run3_tv[:, 0])])
plt.ylim([min(run3_tv[:, 1]), max(run3_tv[:, 1])])
plt.grid(True)
tv3 = cwd + 'tv3.png'
figures.append(tv3)
plt.savefig(tv3)
plt.show()

plt.figure()
plt.plot(run3_tv[:, 0], run3_tv[:, 1]*gpv)
plt.title(tg_title + ' Run 3')
plt.xlabel(xlabel_tv)
plt.ylabel(ylabel_tv)
plt.xlim([min(run3_tv[:, 0]), max(run3_tv[:, 0])])
plt.ylim([min(run3_tv[:, 1]*gpv), max(run3_tv[:, 1]*gpv)])
plt.grid(True)
tg3 = cwd + 'tg3.png'
figures.append(tg3)
plt.savefig(tg3)
plt.show()

plt.figure()
plt.plot(run4_tv[:, 0], run4_tv[:, 1])
plt.title(tv_title + ' Run 4')
plt.xlabel(xlabel_tv)
plt.ylabel(ylabel_tv)
plt.xlim([min(run4_tv[:, 0]), max(run4_tv[:, 0])])
plt.ylim([min(run4_tv[:, 1]), max(run4_tv[:, 1])])
plt.grid(True)
tv4 = cwd + 'tv4.png'
figures.append(tv4)
plt.savefig(tv4)
plt.show()

plt.figure()
plt.plot(run4_tv[:, 0], run4_tv[:, 1]*gpv)
plt.title(tg_title + ' Run 4')
plt.xlabel(xlabel_tv)
plt.ylabel(ylabel_tv)
```

```
plt.xlim([min(run4_tv[:, 0]), max(run4_tv[:, 0])])
plt.ylim([min(run4_tv[:, 1]*gpv), max(run4_tv[:, 1]*gpv)])
plt.grid(True)
tg4 = cwd + 'tg4.png'
figures.append(tg4)
plt.savefig(tg4)
plt.show()

plt.figure()
plt.plot(run1_fft[:, 0], run1_fft[:, 1])
plt.title(fft_title + ' Run 1')
plt.xlabel(xlabel_fft)
plt.ylabel(ylabel_fft)
plt.xlim([min(run1_fft[:, 0]), max(run1_fft[:, 0])])
plt.ylim([min(run1_fft[:, 1]), max(run1_fft[:, 1])])
plt.grid(True)
fft1 = cwd + 'fft1.png'
figures.append(fft1)
plt.savefig(fft1)
plt.show()

plt.figure()
plt.plot(run2_fft[:, 0], run2_fft[:, 1])
plt.title(fft_title + ' Run 2')
plt.xlabel(xlabel_fft)
plt.ylabel(ylabel_fft)
plt.xlim([min(run2_fft[:, 0]), max(run2_fft[:, 0])])
plt.ylim([min(run2_fft[:, 1]), max(run2_fft[:, 1])])
plt.grid(True)
fft2 = cwd + 'fft2.png'
figures.append(fft2)
plt.savefig(fft2)
plt.show()

plt.figure()
plt.plot(run3_fft[:, 0], run3_fft[:, 1])
plt.title(fft_title + ' Run 3')
plt.xlabel(xlabel_fft)
plt.ylabel(ylabel_fft)
plt.xlim([min(run3_fft[:, 0]), max(run3_fft[:, 0])])
plt.ylim([min(run3_fft[:, 1]), max(run3_fft[:, 1])])
plt.grid(True)
fft3 = cwd + 'fft3.png'
figures.append(fft3)
plt.savefig(fft3)
plt.show()

plt.figure()
```

```
plt.plot(run4_fft[:, 0], run4_fft[:, 1])
plt.title(fft_title + ' Run 4')
plt.xlabel(xlabel_fft)
plt.ylabel(ylabel_fft)
plt.xlim([min(run4_fft[:, 0]), max(run4_fft[:, 0])])
plt.ylim([min(run4_fft[:, 1]), max(run4_fft[:, 1])])
plt.grid(True)
fft4 = cwd + 'fft4.png'
figures.append(fft4)
plt.savefig(fft4)
plt.show()

report.include_images(figures)
report.include_file()
report.include_pdf(cwd + 'Lab_10_Outline.pdf', new_section='
   Lab Outline')
report.lend_close()
```

E : Lab Outline

(Attached On Next Page)[9]

[9] A footnote about pdf

Lab 10: Piezoelectric Accelerometer

- Review
  - Vibration of a cantilever
    - Exact method
    - Crude approximation for natural frequency
    - Other approximate methods
      - Rayleigh-Ritz method
      - Assumed modes
      - Finite element
      - etc
  - Piezoelectric accelerometer
    - Equivalent circuit (see spec sheet)
- Equipment
  - Digital Multimeter Pro DMM (link)
  - National Instruments USB 6341(link)
  - Breadboard with triple power supply
  - Sweep/function generator (link)
  - Oscilloscope (link)
  - 741 op amp (link)
    - $V_S^+ = 6$ V
    - $V_S^- = -6$ V
  - OP07 op amp (link)
    - $V_S^+ = 6$ V
    - $V_S^- = -6$ V
  - INA121PA-ND op amp (link)
    - $V_S^+ = 6$ V
    - $V_S^- = -6$ V
  - Resistors:
  - Accelerometer - Metrix Instrument Co.
    - **Please do not directly connect a corroded BNC connector to oscilloscope**

- In our Instrumentation and Controls lab we have several piezoelectric accelerometers. The accelerometers are old and show signs of corrosion. **The objectives of this lab are to: (i) determine experimentally the bending stiffness of your assigned cantilever beam (EI) and (ii) determine if the manufacturers calibration (voltage sensitivity) of your assigned accelerometer remains accurate.**
- Lab report:
  - Your lab report should follow the required lab report structure for this class.
  - In the body of your report, you should build a strong case for your objectives. **The objectives of this lab are to:**
    - **(i) determine experimentally the bending stiffness of your assigned cantilever beam (EI)**
      - Quantify the uncertainty/error in the quantities you obtain
      - Justify your experimental procedure: circuit, sample rate, filter, number of samples, etc.
      - Outline the mathematical theory used in your analysis
      - How does your measurement of stiffness compare to the expected stiffness for the given geometry and material?
    - **(ii) determine if the manufacturers calibration (voltage sensitivity) of your assigned accelerometer remains accurate**
      - What is the manufacturers stated voltage sensitivity?
      - What is the voltage sensitivity you measure?
      - Quantify the uncertainty/error in the quantities you obtain
      - How do these sensitivities compare? Are they within measurement uncertainty?
      - Justify your setup and procedure: circuit, sample rate, filter, number of samples/trials, etc.
      - Clearly state your conclusion
  - Include the following in the Appendix:
    - This lab outline (required)
    - Data you collected during lab (required)
    - Screenshot of each LabView block diagram and/or Matlab scripts used to acquire/process data (required)
    - Responses to the questions throughout the lab outline. (optional)
    - Any other details (data, derivations, calculations, experimental procedure, screen captures etc.)

(The purpose of the Appendix in these reports is to: (i) serve as somewhat of a lab notebook, (ii) demonstrate to the grader what you did during lab, and (iii) hold material that didn't fit into the body of the report but would be necessary to repeat the experiments or justify the claims in the report.)

References